

Objective

Create a synthetic endoscopic-style video showing an opening expanding and contracting, with synchronized pressure labels for exploring visual patterns through machine learning.

Video generation

A Python script generated a stylized tissue-lined opening without using patient images. The opening expands and contracts in a repeating four-second cycle. Camera movement, zoom, and lighting vary separately to introduce visual variation.

The default video contains **180 frames over 12 seconds at 15 frames per second**. The script also supports other durations and frame rates, including 60 seconds at 30 fps.

Pressure dataset

Each frame received a synthetic pressure value based on the opening's simulated size:

$$[P(t)=15+25[s(t)-1]]$$

The equation is:

$$P(t) = 15 + 25 [s(t) - 1]$$

where:

- $P(t)$ is synthetic pressure in mmHg.
- $s(t)$ is the simulated opening's radius multiplier—not a measurement from the image.
- 15 mmHg is the chosen pressure at the reference opening size.
- 25 is the chosen pressure increase per unit of opening scale.

Here, $P(t)$ represents synthetic pressure in mmHg, and $s(t)$ represents the opening's radius multiplier. The scale varies from approximately 0.76 to 1.24, producing pressures of approximately **9–21 mmHg**.

The opening follows:

$$[s(t)=1+0.24\sin\left(\frac{2\pi t}{4}+\phi\right)]$$

The opening itself follows:

$$s(t) = 1 + 0.24 \sin\left(\frac{2\pi t}{4} + \phi\right)$$

Here, ϕ is a reproducible phase offset determined by the random seed. Substituting this into the pressure equation gives approximately:

$$P(t) = 15 + 6 \sin\left(\frac{2\pi t}{4} + \phi\right)$$

The phase offset (ϕ) is determined by a fixed random seed, making the simulation reproducible. Pressure increases as the opening expands and decreases as it contracts.

The dataset also includes an optional noisy sensor-like pressure value and an illustrative flow-rate value. These are simulated quantities, not measurements.

Generated outputs

- Original video and individual unlabeled frames for model inputs.
- A CSV containing frame numbers, timestamps, pressure values, and other simulation labels.
- Labeled copies of each frame displaying pressure, timestamp, and frame number.
- A synchronized video displaying the corresponding pressure values.
- A standalone Python script that reproduces the videos, frames, and dataset.

The labeled images and video are intended for visualization. Training should use the **unlabeled images**, so the model cannot simply read the displayed pressure.

Initial machine-learning experiment

A lightweight regression model was trained to predict synthetic pressure from image pixels. Additional sequences were generated, and entire sequences were assigned to separate training, validation, and test sets.

The model achieved a test mean absolute error of approximately **1.01 mmHg**, compared with **3.82 mmHg** for a baseline that always predicted the training-set average. This evaluation used different sequences from the same synthetic renderer.

Interpretation and limitations

The pressure equation and its constants were chosen for demonstration. They were not derived from measured intrarenal pressures, the teammate's dataset, or a validated tissue-mechanics model.

The work demonstrates a reproducible pipeline for generating images, matching them with numerical labels, and testing whether a model can learn a programmed relationship. It does **not** establish that opening size predicts pressure in real ureteroscopy. Real-world use would require independent validation with synchronized real video and measured pressure data.

CODEX Script:

"""Complete synthetic endoscopic opening video and pressure dataset generator.

Install: python -m pip install numpy pillow imageio-ffmpeg

Run: python ureteroscopy_complete.py

60-second run: python ureteroscopy_complete.py --seconds 60 --fps 30 --out run_60s

No companion scripts or input images are required.

This is a stylized simulation, not validated anatomy or physiology.

"""

```
import argparse
```

```
import csv
```

```
import io
```

```
import json
```

```
import os
```

```
import random
```

```
import shutil
```

```
import struct
```

```
import subprocess
```

```
import zipfile
```

```
from pathlib import Path
```

```
import numpy as np
```

```
from PIL import Image, ImageDraw, ImageFont
```

```
def chunk(tag, data):
```

```
    return tag + struct.pack('<I', len(data)) + data + (b'\0' if len(data) % 2 else b'')
```

```
def render(t, seed, size):
```

```
    rng = np.random.default_rng(seed)
```

```
    phases = rng.uniform(0, 2 * np.pi, 8)
```

```
    y, x = np.mgrid[-1:1:complex(size), -1:1:complex(size)]
```

```
    # Independent camera motion: the camera signal never uses the opening label.
```

```
    zoom = 1 + .08 * np.sin(2 * np.pi * t / 7.3 + phases[0])
```

```
    cx = .055 * np.sin(t * .7 + phases[1])
```

```
    cy = .045 * np.cos(t * .83 + phases[2])
```

```
    xx, yy = (x - cx) / zoom, (y - cy) / zoom
```

```
    angle = np.arctan2(yy, xx) + .04 * np.sin(t * .5 + phases[3])
```

```
    radial = np.sqrt((xx * 1.04) ** 2 + (yy * .94) ** 2)
```

```
    cycle = 2 * np.pi * t / 4 + phases[4]
```

```
    opening = 1 + .24 * np.sin(cycle)
```

```
    speed = .24 * (2 * np.pi / 4) * np.cos(cycle)
```

```
    radius = .31 * opening
```

```
    boundary = radius * (1 + .075 * np.sin(5 * angle + phases[5])  
                        + .045 * np.sin(9 * angle + phases[6]))
```

```

distance = radial - boundary
edge = 1 / (1 + np.exp(-np.clip(distance / .035, -50, 50)))
folds = (.5 + .5 * np.cos(11 * angle + 2 * radial + phases[7]))
folds *= np.exp(-np.maximum(distance, 0) * 2.6)
texture = (np.sin(32 * xx + 8 * np.sin(9 * yy + phases[1]))
           * np.sin(29 * yy + 4 * np.sin(7 * xx)))
# Illustrative ridges and vessel-like detail, fixed in scene coordinates.
vessels = np.exp(-((np.sin(16 * angle + 5 * radial + phases[6])) / .10) ** 2)
light = .87 + .1 * np.sin(t * .39 + phases[0])
shade = (.60 + .32 * folds + .045 * texture - .055 * vessels) * light
tissue = np.stack([205 * shade, 112 * shade, 104 * shade], axis=-1)
cavity = np.stack([19 + 15 * radial, 6 + 5 * radial, 7 + 5 * radial], axis=-1)
rgb = cavity * (1 - edge[..., None]) + tissue * edge[..., None]
rim = np.exp(-(distance / .055) ** 2) * (.5 + .5 * np.cos(angle + 1))
rgb += rim[..., None] * np.array([36, 27, 22])
vignette = np.clip(1.16 - .48 * (x*x + y*y), .2, 1)
field = (x*x + y*y) < .96 ** 2
rgb *= vignette[..., None]
rgb[~field] = 0
image = Image.fromarray(np.uint8(np.clip(rgb, 0, 255)))
mask = Image.fromarray(np.uint8((distance < 0) & field) * 255)
motion = 'near_turning_point' if abs(speed) < .06 else ('expanding' if speed > 0 else
'contracting')
return image, mask, opening, speed, zoom, motion

def find_ffmpeg():
    explicit = os.environ.get('URETEROSCOPY_FFMPEG')
    if explicit and Path(explicit).is_file():
        return explicit
    executable = shutil.which('ffmpeg')
    if executable:
        return executable
    try:
        import imageio_ffmpeg
        return imageio_ffmpeg.get_ffmpeg_exe()
    except (ImportError, AttributeError):
        return None

def font(size):
    try:
        return ImageFont.truetype('C:/Windows/Fonts/arial.ttf', size)
    except OSError:
        return ImageFont.load_default(size=size)

```

```

class VideoWriter:
    """Stream to H.264 MP4 when FFmpeg is present, otherwise Motion-JPEG AVI."""
    def __init__(self, path, fps, size, ffmpeg):
        self.path, self.fps, self.size = path, fps, size
        self.process = None
        self.file = None
        self.count, self.largest, self.offset = 0, 0, 4
        self.index = []
        if ffmpeg:
            self.process = subprocess.Popen([
                ffmpeg, '-y', '-hide_banner', '-loglevel', 'error',
                '-f', 'rawvideo', '-pix_fmt', 'rgb24', '-s', f'{size}x{size}',
                '-r', str(fps), '-i', '-', '-an', '-c:v', 'libx264',
                '-preset', 'medium', '-crf', '21', '-pix_fmt', 'yuv420p',
                '-movflags', '+faststart', str(path)], stdin=subprocess.PIPE)
        else:
            self.file = path.open('wb')
            self.file.write(self.header(0))

    def header(self, total_size):
        avih = struct.pack('<14I', round(1e6/self.fps), self.largest*self.fps,
            0, 16, self.count, 0, 1, self.largest, self.size, self.size, 0, 0, 0, 0)
        strh = struct.pack('<4s4sIIHHIIIIIIhhhh', b'vids', b'MJPG', 0, 0, 0,
            0, 1, self.fps, 0, self.count, self.largest, 0xffffffff, 0,
            0, 0, self.size, self.size)
        strf = struct.pack('<liHH4sliII', 40, self.size, self.size, 1, 24,
            b'MJPG', self.size*self.size*3, 0, 0, 0, 0)
        stream = chunk(b'LIST', b'str'+chunk(b'strh', strh)+chunk(b'strf', strf))
        hdlr = chunk(b'LIST', b'hdlr'+chunk(b'avih', avih)+stream)
        return b'RIFF'+struct.pack('<I', max(0,total_size-8))+b'AVI'
        '+hdlr'+b'LIST'+struct.pack('<I',self.offset)+b'movi'

    def write(self, image):
        if image.size != (self.size, self.size) or image.mode != 'RGB':
            raise ValueError("Video frame must be RGB with the configured dimensions")
        if self.process:
            self.process.stdin.write(image.tobytes())
        else:
            buffer = io.BytesIO()
            image.save(buffer, format='JPEG', quality=92)
            payload = buffer.getvalue()
            block = chunk(b'00dc', payload)
            self.index.append(struct.pack('<4sIII',b'00dc',16,self.offset,len(payload)))
            self.file.write(block)

```

```

        self.offset += len(block)
        self.largest = max(self.largest, len(payload))
        self.count += 1

def close(self):
    if self.process:
        self.process.stdin.close()
        if self.process.wait() != 0:
            raise RuntimeError(f'FFmpeg could not encode {self.path}')
    elif self.file:
        self.file.write(chunk(b'idx1', b''.join(self.index)))
        total = self.file.tell()
        self.file.seek(0)
        self.file.write(self.header(total))
        self.file.close()
        self.file = None

def abort(self):
    if self.process and self.process.poll() is None:
        self.process.kill()
        self.process.wait()
    if self.file:
        self.file.close()
        self.file = None

def label_image(image, pressure, timestamp, frame):
    canvas = Image.new('RGB', (512, 512), '#111820')
    canvas.paste(image.resize((416, 416), Image.Resampling.LANCZOS), (48, 0))
    draw = ImageDraw.Draw(canvas)
    draw.line((16, 420, 496, 420), fill='#527383', width=1)
    draw.text((20, 430), f'Synthetic pressure: {pressure:.2f} mmHg', font=font(28), fill='white')
    draw.text((20, 474), f'Time {timestamp:.2f} s | Frame {frame:05d}',
              font=font(19), fill='#C8D6DF')
    return canvas

def main():
    parser = argparse.ArgumentParser(description=__doc__)

    parser.add_argument('--out', type=Path, default=Path(__file__).resolve().parent/'synthetic_complete')
    parser.add_argument('--seconds', type=int, default=12)
    parser.add_argument('--fps', type=int, default=15)

```

```

parser.add_argument('--size',type=int,default=256)
parser.add_argument('--seed',type=int,default=42)
parser.add_argument('--avi',action='store_true',help='Force AVI even when FFmpeg is
installed')
parser.add_argument('--preview-gif',action='store_true',help='Also create a small labeled GIF
preview')
args = parser.parse_args()
if args.seconds<=0 or args.fps<=0 or not 32<=args.size<=2048 or args.size%2:
    parser.error('Use positive seconds/fps and an even size between 32 and 2048')
destination = args.out.resolve()
# Use a fresh folder to avoid mixing frames from different runs.
if destination.exists() and any(destination.iterdir()):
    parser.error('Output folder is not empty. Choose a new folder with --out.')
plain_dir = destination/'frames'
labeled_dir = destination/'labeled_frames'
plain_dir.mkdir(parents=True,exist_ok=True)
labeled_dir.mkdir(parents=True,exist_ok=True)
ffmpeg = None if args.avi else find_ffmpeg()
extension = '.mp4' if ffmpeg else '.avi'
plain_path = destination/('synthetic_opening'+extension)
labeled_path = destination/('synthetic_opening_with_pressure'+extension)
writers = []
rng = random.Random(20260927)
noise = 0.0
rows, preview = [], []
preview_step = max(1,round(args.fps/5))
try:
    writers.append(VideoWriter(plain_path,args.fps,args.size,ffmpeg))
    writers.append(VideoWriter(labeled_path,args.fps,512,ffmpeg))
    for frame in range(args.seconds*args.fps):
        t = frame/args.fps
        image, _, opening, speed, zoom, motion = render(t,args.seed,args.size)
        opening, speed = round(float(opening),6),round(float(speed),6)
        # Arbitrary illustrative mapping shared with the previous dataset.
        pressure = 15+25*(opening-1)
        noise = .88*noise+rng.gauss(0,.12)
        labeled = label_image(image,pressure,t,frame)
        plain_name = f'frames/frame_{frame:05d}.png'
        labeled_name =
f'labeled_frames/frame_{frame:05d}_pressure_{pressure:.2f}_mmHg.png'
        image.save(destination/plain_name)
        labeled.save(destination/labeled_name)
        writers[0].write(image)
        writers[1].write(labeled)

```

```

        if args.preview_gif and frame%preview_step==0:
            preview.append(labeled.resize((384,384),Image.Resampling.LANCZOS))
        rows.append(dict(sequence_id=f'seed_{args.seed}',frame=frame,
            timestamp_s=f'{t:.6f}',fps=args.fps,video_file=plain_path.name,

labeled_video_file=labeled_path.name,image=plain_name,labeled_image=labeled_name,
            opening_scale=f'{opening:.6f}',opening_change_per_s=f'{speed:.6f}',motion=motion,
            camera_zoom=f'{zoom:.6f}',synthetic_intrarenal_pressure_mmHg=f'{pressure:.6f}',
            displayed_pressure_mmHg=f'{pressure:.2f}',
            synthetic_pressure_change_mmHg_s=f'{25*speed:.6f}',
            synthetic_sensor_pressure_mmHg=f'{pressure+noise:.6f}',
            synthetic_flow_rate_ml_min=f'{30+1.5*(pressure-15):.6f}',
            is_synthetic_ground_truth=True,relationship_model='illustrative_linear_opening_v1'))
        if frame%args.fps==0:
            print(f'Generating {frame//args.fps+1}/{args.seconds} seconds',flush=True)
        for writer in writers:
            writer.close()
    except BaseException:
        for writer in writers:
            writer.abort()
        raise
    with (destination/'synthetic_pressure.csv').open('w',newline="",encoding='utf-8') as f:
        writer=csv.DictWriter(f,fieldnames=rows[0])
        writer.writeheader()
        writer.writerows(rows)
    metadata=dict(duration_s=args.seconds,fps=args.fps,frames=len(rows),size=args.size,

scene_seed=args.seed,sensor_seed=20260927,presure_equation='15+25*(opening_scale-1)',
    pressure_units='mmHg (illustrative simulation
only)',nominal_pressure_range_mmHg=[9,21],
    flow_equation='30+1.5*(pressure-15)',
    noise_equation='e[t]=0.88*e[t-1]+Normal(0,0.12), initial e=0',
    video_format=extension,presure_label='synthetic_intrarenal_pressure_mmHg',
    physiological_validation=False)
    (destination/'metadata.json').write_text(json.dumps(metadata,indent=2),encoding='utf-8')
    if preview:
        durations=[round(1000*min(preview_step,len(rows)-i)/args.fps)
            for i in range(0,len(rows),preview_step)]
        preview[0].save(destination/'labeled_preview.gif',save_all=True,
            append_images=preview[1:],duration=durations,loop=0)
    with zipfile.ZipFile(destination/'labeled_frames.zip','w',zipfile.ZIP_DEFLATED) as archive:
        for row in rows:
            archive.write(destination/row['labeled_image'],row['labeled_image'])
        archive.write(destination/'synthetic_pressure.csv','synthetic_pressure.csv')

```

```

(destination/'README.txt').write_text(
    'SYNTHETIC VIDEO AND PRESSURE DATA\n\n'
    'frames/: original unlabeled training images.\n'
    'labeled_frames/: pressure, timestamp, and frame number for viewing only.\n'
    'synthetic_opening: original video.\n'
    'synthetic_opening_with_pressure: synchronized labeled video.\n'
    'synthetic_pressure.csv: one row per frame, with both image paths.\n'
    'labeled_frames.zip: labeled images plus the CSV.\n'
    'metadata.json: simulation settings.\n\n'
    'Use original images as inputs and synthetic_intrarenal_pressure_mmHg as targets.\n'
    'Do not train on labeled images or give the model simulation labels/timestamps as inputs.\n'
    'Pressure is an assumed linear mapping from rendered opening size, not a
measurement.\n'
    'Noisy sensor pressure is a simulated observation; noise is not visible in the images.\n'
    'Flow is an illustrative companion label with no independent visual effect.\n'
    'The renderer is stylized and is not anatomically or physiologically validated.\n'
    'Keep whole sequences in separate dataset splits. A new seed changes phases
only.\n',encoding='utf-8')
print(f'COMPLETE: {len(rows)} frame pairs and pressure rows.\nResults: {destination}')
print(f'Original video: {plain_path.name}\nLabeled video: {labeled_path.name}')
if not ffmpeg:
    print('AVI output used. For MP4 install: python -m pip install imageio-ffmpeg')

if __name__ == '__main__':
    main()

```